



Hệ Sinh Thái Công Cụ AI Coding

Phần I · Nền Tảng · Chương 3

Bản Đồ Toàn Cảnh

Nguyên lý hoạt động · Phân loại đa chiều và chiến lược chọn tool cho đồ án

Playbook SDD & ADD

Mục Tiêu Chương Này

Chương 3 — Giới Thiệu

- Hiểu nguyên lý hoạt động của 3 triết lý thiết kế AI coding tools — không chỉ tính năng hiện tại
- Phân biệt Model-Locked vs Model-Agnostic và tác động đến chi phí, bảo mật, linh hoạt
- Đánh giá chi tiết 6+ tools với bảng Snapshot thống nhất
- Sử dụng Decision Matrix để chọn tool phù hợp cho từng bối cảnh
- Hiểu về Data Privacy, Local LLM, và Enterprise compliance
- Xây dựng chiến lược multi-tool cho nhóm sinh viên

⚠️ **Lưu Ý Về Tính Thời Sự** — Công nghệ AI coding thay đổi rất nhanh. Chương này tập trung vào NGUYÊN LÝ hoạt động và TRIẾT LÝ thiết kế — những thứ thay đổi chậm hơn. Giá cả và tính năng chi tiết có thể đã thay đổi, nhưng cách phân loại và nguyên lý so sánh vẫn áp dụng được.

Nội Dung Chương 3

Chương 3 — Mục Lục

1

3.1 — Phân Loại Theo Nguyên Lý

3 triết lý tích hợp · Model-Locked vs Agnostic · RAG vs Long Context

2

3.2 — Đánh Giá Chi Tiết Từng Tool

Claude Code · Cursor · Copilot · Codex CLI · Kiro · Cline · Aider

3

3.3 — Bảng Quyết Định Chọn Tool

Decision Matrix 6 chiều · Data Privacy · Phân tích chi phí TCO

4

3.4 — Chiến Lược Multi-Tool

Cursor + Claude Code · Combo \$0 · Khuyến nghị nhóm 5 SV



Phân Loại AI Coding Tools Theo Nguyên Lý Hoạt Động

Phần 3.1

Toàn bộ hệ sinh thái AI coding chỉ có 3 triết lý thiết kế cơ bản.

Mọi tool là một biến thể của 1 trong 3 triết lý này.

Hiểu được điều này, bạn sẽ không bao giờ bị "lạc" khi tool mới xuất hiện.

Triết Lý A: IDE-Native

3.1.1 — Trục 1: Triết Lý Tích Hợp

"AI sống trong editor của bạn" — AI có quyền truy cập tự nhiên vào mọi thứ: file đang mở, project structure, terminal output, git history, và cursor position.

Ưu Điểm

- Flow state: Không cần chuyển ứng dụng. AI xuất hiện đúng chỗ bạn cần
- Context tự nhiên: AI thấy những gì bạn thấy. Không cần paste code
- Làm quen nhanh: Biết VS Code → Cursor chỉ mất 5 phút
- Codebase indexing: Index toàn bộ dự án để AI hiểu kiến trúc

Nhược Điểm

- Lock-in vào IDE: Phải dùng IDE của họ. Cursor là VS Code fork
- Autonomy giới hạn: Agent mode kém terminal-native vì thiếu full shell access
- Large refactors khó: Dự án 100+ files có thể chậm hoặc mất context

Tools Đại Diện

Cursor · Windsurf · Amazon Kiro · Zed AI

Triết Lý B: Plugin/Extension

3.1.1 — Trục 1: Triết Lý Tích Hợp

"AI là phần mở rộng của editor bạn" — Không tạo IDE mới, thêm AI vào IDE đã dùng. Bạn giữ nguyên môi trường làm việc hiện tại.

Ưu Điểm

- Không đổi IDE: Giữ nguyên workflow. Đặc biệt quan trọng với JetBrains/Vim
- Ecosystem rộng: Copilot chạy trên mọi IDE lớn. Không ai bị bỏ rơi
- Enterprise adoption: Dễ được IT department chấp nhận vì chỉ là extension

Nhược Điểm

- Context hạn chế: Chỉ truy cập được những gì extension API cho phép
- Agentic yếu hơn: Hầu hết là chat + autocomplete, không có full agent mode
- Phụ thuộc IDE updates: Khi IDE thay đổi API, plugin có thể hỏng

Tools Đại Diện

GitHub Copilot · Cline · Continue.dev · Cody (Sourcegraph)

Triết Lý C: Terminal-Native Agentic

3.1.1 — Trục 1: Triết Lý Tích Hợp

"AI là đồng nghiệp làm việc qua terminal" — AI không chỉ gợi ý code mà TỰ LÀM: đọc codebase → lên plan → edit files → chạy tests → fix errors → commit.

Ưu Điểm

- Autonomy cao nhất: Đọc/ghi files, chạy commands, install packages
- Codebase awareness: Đọc toàn bộ project, hiểu kiến trúc tổng thể
- Multi-file editing: Sửa hàng chục files trong 1 task — refactor toàn project
- IDE-agnostic: Hoạt động với bất kỳ editor nào vì không cần editor

Nhược Điểm

- Learning curve: Terminal-based workflow không quen với nhiều sinh viên
- Risk cao hơn: Agent có thể chạy lệnh nguy hiểm nếu không có guard rails
- Không visual feedback: Không thấy diff view tự động (trừ khi dùng với IDE extension)
- Chi phí: Dùng nhiều tokens hơn → đắt hơn per task

Tools Đại Diện

Claude Code · OpenAI Codex CLI · Gemini CLI · Aider

Bảng 3.1: So Sánh 3 Triết Lý Tích Hợp

3.1.1 — Bảng So Sánh

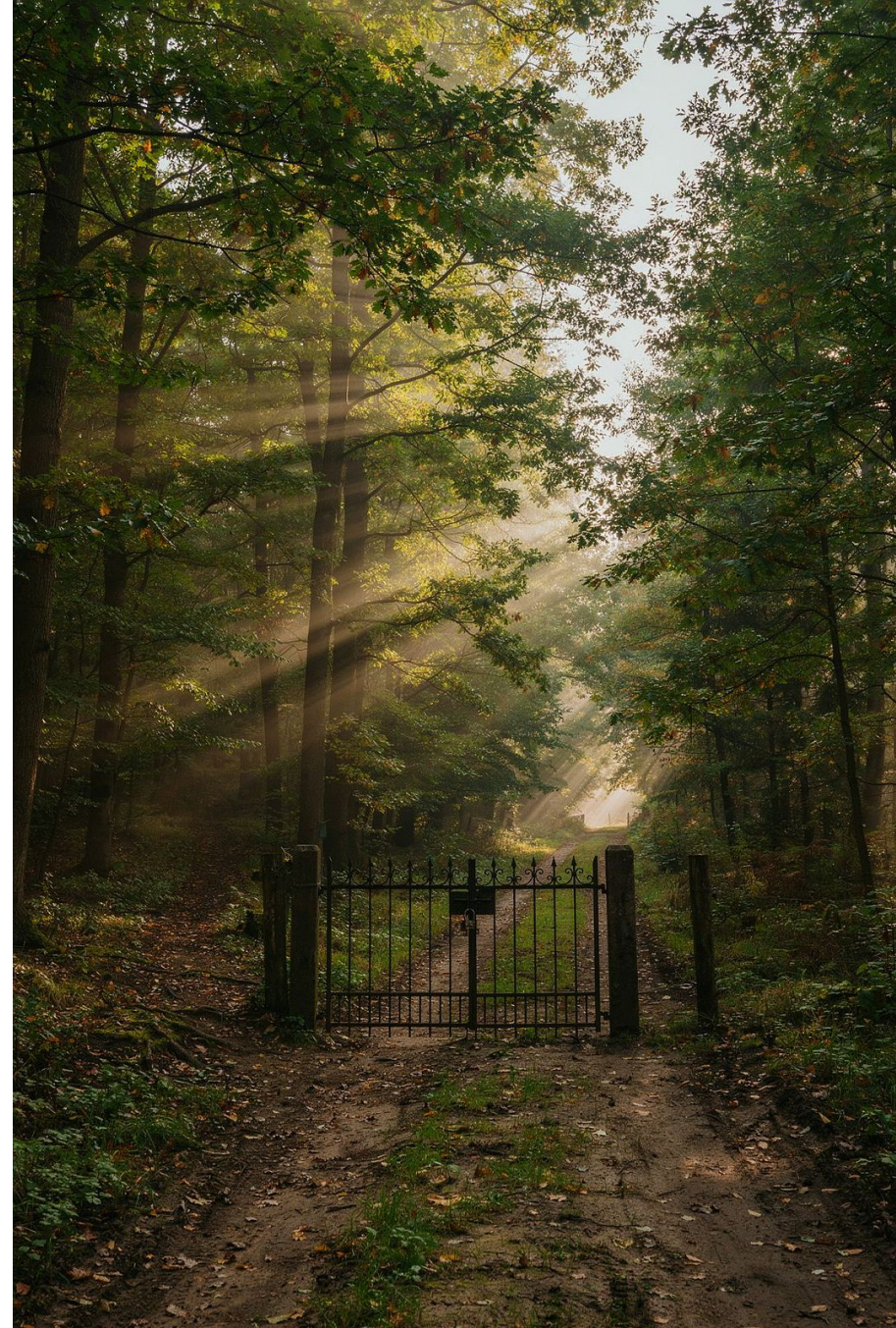
Tiêu chí	IDE-Native	Plugin/Extension	Terminal-Native
Phép ẩn dụ	Kiến trúc sư tích hợp	Cố vấn bên cạnh	Đồng nghiệp tự chủ
Context depth	Cao (file + project + terminal)	Trung bình (file + partial project)	Rất cao (toàn bộ filesystem)
Autonomy	Trung bình (Agent mode)	Thấp (Chat + autocomplete)	Rất cao (full shell access)
Learning curve	Thấp (VS Code familiar)	Rất thấp (cài extension)	Trung bình (terminal skills)
Workflow change	Vừa (editor mới)	Không (đồ cũ)	Lớn (terminal-first)
Multi-file editing	Tốt	Hạn chế	Xuất sắc
Phù hợp cho	Daily coding, vừa + nhỏ	Mọi IDE, team lớn	Complex tasks, refactors
Ví dụ tool	Cursor, Windsurf, Kiro	Copilot, Cline	Claude Code, Codex CLI

Model-Agnostic vs Model-Locked

3.1.2 — Trục 2

Trục phân loại thứ hai cực kỳ quan trọng mà nhiều người bỏ qua.

Ảnh hưởng trực tiếp đến 3 yếu tố: chi phí · bảo mật · linh hoạt tương lai



Model-Locked vs Model-Agnostic

3.1.2 — Model Openness

Model-Locked ("Đóng")

Chỉ dùng model của nhà cung cấp. Không thể chọn model khác, không chạy local.

- Claude Code: Chỉ Anthropic (Opus 4.6, Sonnet 4.6)
- Amazon Kiro: Amazon Bedrock (Claude + Amazon models)
- Copilot (trước): Ban đầu chỉ OpenAI models

✓ Tối ưu sâu, đơn giản, output nhất quán

× Vendor lock-in, chi phí cố định, data luôn lên cloud

Model-Agnostic ("Mở")

Đổi model như đổi ống kính. Claude cho task phức tạp, GPT-4o cho task nhanh, Llama local cho code nhạy cảm.

- Cursor: GPT-4o, Claude, Gemini, custom API key
- Cline: Anthropic, OpenAI, Google, Ollama (local)
- Aider: 30+ models. Ollama cho 100% local
- Continue.dev: Mọi provider kể cả local

✓ Tiết kiệm 50-70%, bảo mật, không lock-in

× Phức tạp hơn, cần quản lý nhiều API keys

Ví Dụ: Tiết Kiệm Chi Phí Với Model-Agnostic

3.1.2 — Tối Ưu Chi Phí

Nhóm 5 SV · 20 tuần · 8h/ngày × 5 ngày/tuần

\$2,500

PA1: Claude Code only

(Model-Locked)

\$700

PA2: Cursor + API

(Model-Agnostic)

\$100

PA3: Cline + Ollama

(Local-first)

PA2 vs PA1

Tiết kiệm 72% — $5 \times \$20/\text{tháng Cursor} + \sim \$200 \text{ API} = \$700 \text{ tổng}$

PA3 vs PA1

Tiết kiệm 96% — nhưng cần máy mạnh, model local yếu hơn cloud

Quy Tắc Chọn Nhanh

- Ngân sách hạn hẹp + máy mạnh → Agnostic + Local (Cline + Ollama)
- Cần chất lượng cao nhất → Model-Locked (Claude Code)
- Team lớn + nhất quán → Model-Locked (mọi người cùng model)
- Bảo mật cao → Model-Agnostic + Local LLM

Bảng 3.2: Ma Trận 2 Chiều

3.1.2 — Bảng So Sánh

Triết lý tích hợp × Model openness

	Model-Locked (Đóng)	Model-Agnostic (Mở)
IDE-Native	Kiro (Amazon models) Windsurf (hỗn hợp)	Cursor (GPT/Claude/Gemini/local) Zed AI (multi-provider)
Plugin/Extension	Copilot (chủ yếu OpenAI)	Cline (mọi provider + Ollama) Continue.dev (mọi provider + local)
Terminal-Native	Claude Code (Anthropic)	Aider (30+ models + Ollama) Codex CLI (OpenAI, open-source)

Cách Tool Xử Lý Context RAG vs Long Context

3.1.3 — Trục 3

Context là tài nguyên hữu hạn. Các tools xử lý giới hạn này bằng 2 chiến lược khác nhau.

Hiểu sự khác biệt giúp bạn dùng tool hiệu quả hơn.



RAG vs Long Context Window

3.1.3 — Context Strategies

Chiến Lược 1: RAG

Index codebase → Vector DB → Query tìm chunks liên quan → Gửi ít tokens cho LLM. Giống như Google: không đọc toàn bộ internet, chỉ trả kết quả relevant.

- Ưu: Tiết kiệm tokens, nhanh, hoạt động codebase lớn
- Nhược: Có thể miss context quan trọng. "AI không biết những gì không được cho thấy."

Tools dùng RAG

Cursor · Copilot · Continue.dev

Chiến Lược 2: Long Context

Gửi nhiều files trực tiếp vào context window. AI "thấy" tất cả cùng lúc. Giống trải hết giấy tờ lên bàn lớn để nhìn toàn cảnh.

- Ưu: AI hiểu toàn cảnh, không miss context. Tốt cho refactoring, architecture
- Nhược: Tốn nhiều tokens, đắt, chậm hơn

Gemini 2.5 Pro (1M tokens) → ~600K dòng code

Claude Opus (200K tokens) → ~130K dòng code

Tools dùng Long Context

Claude Code · Kiro · Aider

📌 Thực Tế: Hầu Hết Tools Dùng HYBRID — **Claude Code**: RAG tìm files liên quan → load toàn bộ vào long context · **Cursor**: index RAG, cho phép @file để chỉ định cụ thể



Đánh Giá Chi Tiết Từng Tool

Phần 3.2

6 tools chính được đánh giá theo cùng một bộ tiêu chí để so sánh công bằng.

Mỗi tool có bảng Snapshot thống nhất, sau đó là phân tích sâu về nguyên lý và use case.

⚠️ ⚠️ Lưu Ý Về Giá — Giá ghi nhận tại Q1/2026. Kiểm tra website chính thức trước khi quyết định.

Claude Code (Anthropic)

3.2.1 — Snapshot

Triết lý	Terminal-Native Agentic
Model	Model-Locked: Claude Opus 4.6, Sonnet 4.6
Context Handling	Hybrid: Agent tự chọn files (search) + load vào 200K context window
Execution Power	Rất cao: đọc/ghi files, chạy terminal, git, install packages, sub-agents
Agentic Capability	Plan → Execute → Observe → Adjust. Extended Thinking. Multi-agent spawning.
Chi phí	Pro \$20/mo (hạn chế) · Max \$100/mo (5x) · Max \$200/mo (20x) · API: pay-per-use
Free tier	Không (cần subscription hoặc API key)
IDE Integration	CLI + VS Code extension + Desktop app + Browser
MCP Support	Có — kết nối Google Drive, Jira, Slack, custom tools
Data Privacy	Code gửi lên Anthropic servers. ZDR (Zero Data Retention) cho API.
Config files	CLAUDE.md (project memory) · slash commands · hooks
Open-source	Không (proprietary)
Điểm mạnh nhất	Deep reasoning · multi-file refactoring · "hiểu" cả codebase
Điểm yếu nhất	Chi phí cao · model-locked · learning curve terminal

Claude Code: Nguyên Lý & Khi Nào Chọn

3.2.1 — Claude Code

Vòng Đời Một Task

1. Bạn mô tả mục tiêu → agent lên plan (Extended Thinking)
2. Đọc CLAUDE.md → biết tech stack và conventions
3. Tự tìm và đọc tất cả files liên quan
4. Hiển thị plan → bạn review và approve
5. Agent tự thực hiện: sửa files, chạy tests, fix lỗi
6. Báo cáo kết quả: "Done. 7 files changed, 2 created."

Khi Nào Chọn Claude Code

- Task phức tạp, multi-file: Refactoring lớn, thay đổi kiến trúc, migrate patterns
- Cần deep reasoning: Debug lỗi logic phức tạp, phân tích architecture
- Greenfield projects: Scaffold toàn bộ dự án từ spec
- Thoải mái với terminal: CLI-first workflow

Agent không chỉ code mà còn install dependencies, áp dụng cho nhiều files, viết tests, và chạy tests — tất cả trong 1 task. Đây là sự khác biệt cốt lõi so với chat-based tools.

Triết lý	IDE-Native (VS Code fork)
Model	Model-Agnostic: GPT-4o, Claude Sonnet/Opus, Gemini, custom via API key
Context Handling	Hybrid: Codebase indexing (RAG) + @file/@folder cho manual selection
Execution Power	Trung bình-cao: Agent mode chạy terminal, nhưng trong IDE sandbox
Agentic Capability	Agent Mode (plan + multi-file edit + terminal). Background agents (beta).
Chi phí	Free (hạn chế) · Pro \$20/mo · Business \$40/mo
Free tier	Có (giới hạn số requests/tháng)
IDE Integration	Là IDE (VS Code fork). Cũng có ở Cursor.com web.
MCP Support	Có (từ 2025)
Data Privacy	Privacy Mode: code không được lưu trữ. SOC 2 certified.
Config files	.cursorrules (rules) · .cursorignore (exclude files)
Điểm mạnh nhất	Flow state · autocomplete nhanh · familiar UX (VS Code)
Điểm yếu nhất	Large refactors không bằng Claude Code · giá thay đổi thường xuyên

Cursor: Tại Sao "Mượt" & Khi Nào Chọn

3.2.2 — Cursor

.cursorrules — Constitution Cho IDE

Hướng dẫn Cursor về conventions, patterns, constraints của dự án — tương tự AGENTS.md.

.cursorrules

- Use camelCase for functions, PascalCase for types
- Error handling: always use AppError class
- Database: use Prisma ORM, never raw SQL
- Architecture: Controller → Service → Repository
- Tests: Jest, co-locate in **__tests__** folders
- NEVER use `any` type
- ALWAYS validate input with zod

Khi Nào Chọn Cursor

- Daily coding: Autocomplete nhanh, small-medium tasks hàng ngày
- Đã dùng VS Code: Chuyển sang mất 5 phút, giữ mọi extensions
- Cần nhiều models: Claude cho tasks khó, GPT-4o mini cho tasks nhanh
- Ngân sách hạn chế: \$20/tháng Pro là mức giá phải chăng cho sinh viên

GitHub Copilot

3.2.3 — Snapshot

Triết lý	Plugin/Extension + đang chuyển dần sang Agentic
Model	Hỗn hợp: chủ yếu GPT-4o, đang mở cho Claude và Gemini
Context Handling	RAG (embeddings của repo) + @workspace cho broad context
Execution Power	Trung bình: Agent mode (mới), Copilot Workspace cho GitHub-centric
Agentic Capability	Agent Mode · Copilot Workspace · Agent HQ (multi-agent, beta 02/2026)
Chi phí	Free (hạn chế) · Individual \$10/mo · Business \$19/mo · Enterprise \$39/mo
Free tier	Có — Free cho verified students và open-source maintainers
IDE Integration	VS Code · JetBrains · Vim/Neovim · Xcode · Visual Studio
MCP Support	Đang phát triển (Agent HQ)
Data Privacy	Business/Enterprise: code KHÔNG dùng training. Individual: có thể được dùng.
Config files	.github/copilot-instructions.md
Điểm mạnh nhất	Ecosystem GitHub · giá rẻ · miễn phí cho SV · enterprise features
Điểm yếu nhất	Agentic capability đang chuyển tiếp · chưa mạnh bằng Claude Code/Cursor

📄 Copilot Cho Đồ Án Sinh Viên — Cả nhóm có GitHub Education → Copilot miễn phí (\$0). Dùng Copilot cho daily coding + Claude Code API (pay-per-use) cho tasks phức tạp.

OpenAI Codex CLI

3.2.4 — Snapshot

Triết lý	Terminal-Native Agentic · Open-Source
Model	Chủ yếu OpenAI (o3, o4-mini, GPT-4.1). Hỗ trợ providers khác qua config.
Context Handling	File reading + AGENTS.md layered system
Execution Power	Cao: sandboxed execution, network-disabled by default (an toàn)
Agentic Capability	Plan + execute trong sandbox. 3 modes: suggest · auto-edit · full-auto.
Chi phí	Tool miễn phí (open-source). Trả API usage cho OpenAI.
Free tier	Tool free, cần OpenAI API key (\$)
Data Privacy	Sandbox mặc định: network disabled, chỉ truy cập project folder
Config files	AGENTS.md (layered: global → project → directory)
Open-source	Có — Apache 2.0 license
Điểm mạnh nhất	Open-source · sandbox an toàn · AGENTS.md native · tùy chỉnh cao
Điểm yếu nhất	Mới · ít mature hơn Claude Code · cộng đồng nhỏ hơn

Codex CLI là birthplace của AGENTS.md — format giờ đã được 60K+ projects sử dụng và donate cho Linux Foundation.

Amazon Kiro

3.2.5 — Snapshot

Triết lý	IDE-Native với SDD built-in
Model	Model-Locked: Amazon Bedrock (Claude + Amazon models)
Context Handling	Long context (Gemini 1M tokens qua Bedrock)
Execution Power	Trung bình: IDE-based execution, DevOps automation
Agentic Capability	SDD workflow: requirements.md → design.md → tasks.md → implement
Chi phí	Free (beta) tại thời điểm viết, giá chưa công bố chính thức
IDE Integration	Là IDE (Code OSS fork, giống VS Code)
Data Privacy	AWS compliance (SOC 2, HIPAA eligible với Bedrock)
Config files	requirements.md · design.md · tasks.md (định dạng EARS)
Open-source	Không (Amazon proprietary)
Điểm mạnh nhất	SDD tích hợp sẵn trong IDE · EARS notation · spec-first workflow
Điểm yếu nhất	Mới (07/2025) · performance chưa ổn định · Amazon ecosystem lock-in

Kiro là tool duy nhất tích hợp SDD workflow trực tiếp vào IDE — "training wheels" tốt nhất cho người mới học SDD. Đáng theo dõi nhưng chưa nên dựa hoàn toàn vào.

Cline · Continue.dev · Aider

3.2.6 — Model-Agnostic Tools Nổi Bật

3 tools chia sẻ triết lý chung: dùng bất kỳ model nào, kể cả local models

Tiêu chí	Cline	Continue.dev	Aider
Triết lý	Plugin, Agnostic, Agentic	Plugin (VS Code + JetBrains)	Terminal-Native, Agnostic
Models	Anthropic, OpenAI, Google, Ollama, OpenRouter	Mọi providers + Ollama + LM Studio + custom	30+ models: GPT, Claude, Gemini, Llama, DeepSeek, Qwen...
Chi phí	Miễn phí (open-source)	Miễn phí (Apache 2.0)	Miễn phí (open-source)
Agentic	Có: đọc/ghi files, terminal, browser automation	Hạn chế: chat + autocomplete (đang phát triển)	Có: edit files, chạy tests, git integration
Đặc biệt	Structured modes (plan/code/review), token tracking chi tiết	JetBrains support, local-first design	Leaderboard benchmark models để chọn tối ưu
Local LLM	Có — Ollama, LM Studio	Có — thiết kế đặc biệt cho local-first	Có — Ollama native, LM Studio

Setup Local LLM với Cline + Ollama

3.2.6 — Setup Thực Hành

Dùng AI coding 100% local — code không bao giờ rời máy bạn.

```
# Bước 1: Cài Ollama
# macOS: brew install ollama
# Linux: curl -fsSL https://ollama.com/install.sh | sh
# Windows: tải installer từ ollama.com

# Bước 2: Tải model coding tốt nhất cho local
ollama pull qwen2.5-coder:14b # 14B params, tốt cho coding
ollama pull deepseek-coder-v2:16b # Alternative tốt
ollama pull llama3.1:8b      # Nhẹ hơn, cho máy yếu

# Bước 3: Chạy thử
ollama run qwen2.5-coder:14b "Viết function hello world bằng TypeScript"

# Bước 4: Cấu hình Cline trong VS Code
# Settings > Cline > API Provider: Ollama
# Model: qwen2.5-coder:14b | Base URL: http://localhost:11434

# Yêu cầu phần cứng tối thiểu:
# 14B model: 16GB RAM, hoặc GPU 10GB VRAM (RTX 3060)
# 8B model: 8GB RAM, hoặc GPU 6GB VRAM
```

⚠️ ⚠️ Giới Hạn Của Local LLM — Model local (8B-14B) YẾU HƠN NHIỀU so với Claude Opus hay GPT-4o. Phù hợp: autocomplete, small tasks, bảo mật · KHÔNG phù hợp: complex reasoning, multi-file refactoring, architecture. Chiến lược tối ưu: local cho daily work (80%) + cloud API cho tasks khó (20%)

Cùng 1 Task · 3 Workflows Khác Nhau

3.2.7 — So Sánh Thực Tế

Task: "Thêm full-text search tiếng Việt cho dự án thư viện sách"

Cursor (IDE-Native) · ~15 phút

1. Mở project → Cursor tự index (30s)
 2. Cmd+L → Agent Mode → prompt với @files được chỉ định
 3. Cursor hiển thị plan trong chat
 4. Approve → sửa files với diff view inline
 5. Chạy tests thủ công trong terminal
- ✓ Diff view trực quan · × Phải chỉ định @files

Claude Code (Terminal) · ~10 phút

1. `$ claude` → mô tả task đơn giản
 2. Tự đọc CLAUDE.md → biết tech stack
 3. Tự tìm tất cả files liên quan
 4. Hiển thị plan → bạn approve
 5. Tự chạy migration + tests + fix lỗi
- ✓ Tự động hoàn toàn · × Không diff view

Copilot (Plugin) · ~30 phút

Hỏi → nhận code snippet → tự copy vào file → tự tạo migration → tự chạy tests → tự fix errors → tự test. Kiểm soát từng dòng nhưng chậm 2-3x.

Bảng 3.5: So Sánh 3 Workflows Cho Cùng 1 Task

3.2.7 — Bảng So Sánh

Tiêu chí	Cursor	Claude Code	Copilot
Thời gian	~15 phút	~10 phút	~30 phút
Files tự xử lý	Phải chỉ định @files	Tự tìm	Từng file manually
Chạy tests	Thủ công	Tự động	Thủ công
Fix errors	Chat lại	Tự động fix + re-run	Thủ công
DB migration	Agent mode có thể tạo file	Tự tạo + chạy	Bạn tự làm
Trực quan	Diff view inline ★	Git diff (terminal)	Xem code trực tiếp
Kiểm soát	Từng file	Từng task	Từng dòng
Phù hợp cho task này?	Tốt	Rất tốt	Được nhưng chậm

📌 Insight Từ So Sánh — **Claude Code** nhanh nhất cho task multi-file vì TỰ ĐỘNG: tìm files, chạy migration, chạy tests, fix errors. · **Cursor** tốt nhất về trải nghiệm vì diff view trực quan. · **Combo tối ưu**: Claude Code cho setup ban đầu, Cursor cho tinh chỉnh hàng ngày.

Bug: Search "nguyễn" Trả Về 0 Kết Quả

3.2.8 — Cùng 1 Bug · 3 Cách Debug

Nguyên nhân: unaccent extension chưa được cài trong PostgreSQL

Cursor

1. Highlight code search → Cmd+L → hỏi về bug
2. Cursor gợi ý: "Kiểm tra `SELECT unaccent('nguyễn')`"
3. Bạn tự chạy SQL trong psql hoặc DBeaver
4. Xác nhận lỗi → tự tạo migration

Claude Code

1. 1 prompt: "*Search 'nguyễn' trả về 0 kết quả. Debug.*"
2. Tự chạy `psql -c "SELECT unaccent('nguyễn')"`
3. Phát hiện: `ERROR: function unaccent does not exist`
4. Tự tạo migration → chạy → test lại → báo cáo kết quả

Copilot

Hỏi → giải thích cần unaccent extension
→ bạn tự kiểm tra psql → tự tạo migration
→ tự chạy → tự test. Chỉ giải thích, bạn làm hết.

Claude Code giải quyết bug end-to-end (tìm nguyên nhân + fix + verify) chỉ với 1 prompt. Mức độ autonomy khác nhau rõ ràng giữa 3 triết lý.

AGENTS.md · CLAUDE.md · .cursorrules

3.2.9 — Hệ Sinh Thái Config Files

File	Tool đọc	Mục đích	Scope
AGENTS.md	Codex CLI, Cursor, Cline, mọi agent	Hướng dẫn chung cho MỌI AI tool	Universal
CLAUDE.md	Claude Code	Memory + context cho Claude	Claude only
.cursorrules	Cursor	Rules cho Cursor AI	Cursor only
.github/copilot-instructions.md	GitHub Copilot	Hướng dẫn cho Copilot	Copilot only
.continuerc.json	Continue.dev	Config cho Continue	Continue only
.aider.conf.yml	Aider	Config cho Aider	Aider only

Chiến Lược Tối Ưu: AGENTS.md Là Nguồn Sự Thật

Thay vì duy trì nhiều files khác nhau, viết AGENTS.md là nguồn chính (universal standard), sau đó generate các file tool-specific từ nó.

CLAUDE.md = AGENTS.md + Claude-specific · .cursorrules = AGENTS.md reformatted cho Cursor · Dùng pre-commit hook để luôn sync

Bảng Quyết Định Chọn Tool Theo Tình Huống

Phần 3.3

Không có tool nào tốt nhất cho mọi tình huống — chỉ có tool phù hợp nhất cho bối cảnh của bạn.

Decision Matrix 6 chiều · Data Privacy 3 mức · Phân tích TCO



Decision Matrix: Quy Mô & Ngân Sách

3.3.1 — Decision Matrix

Câu 1: Quy Mô Dự Án?

Quy mô	Mô tả	Tool khuyên nghị
Nhỏ (<5K LOC)	Prototype, hackathon, side project	Cursor hoặc Copilot — nhanh, đơn giản
Vừa (5-50K LOC)	Đồ án SE, startup MVP	Cursor + Claude Code (combo)
Lớn (>50K LOC)	Enterprise, legacy system	Claude Code (agentic) + Copilot (team)

Câu 2: Ngân Sách?

Ngân sách	Lựa chọn tối ưu
\$0 (miễn phí)	Copilot (GitHub Education) + Cline (open-source) + Ollama (local)
\$10-20/người/tháng	Cursor Pro (\$20) hoặc Copilot Individual (\$10)
\$20-50/người/tháng	Cursor Pro + Claude Code API (pay-per-use)
>\$100/người/tháng	Claude Code Max (\$100-200) cho full power

Decision Matrix: Kỹ Năng & Loại Dự Án

3.3.1 — Decision Matrix

Câu 3: Kỹ Năng Team?

Kỹ năng	Tool khuyên nghị	Lý do
Mới học lập trình	Cursor + Copilot	UI quen thuộc, learning curve thấp nhất
Biết cơ bản, chưa pro	Cursor (Agent Mode)	Agent hỗ trợ nhưng vẫn trong IDE quen
Thoải mái với terminal	Claude Code hoặc Aider	Tận dụng full agentic power
Hỗn hợp (nhóm 5 người)	Cursor (chung) + Claude Code (lead)	Mọi người dùng Cursor, lead dùng CC cho tasks khó

Câu 4: Loại Dự Án?

Loại dự án	Tool khuyên nghị	Lý do
Greenfield (từ đầu)	Claude Code + Spec Kit	Scaffold nhanh từ spec
Brownfield (mở rộng)	Cursor + Claude Code	Cần hiểu codebase hiện tại
Data pipeline	Claude Code (terminal-first)	Cần chạy scripts, process data
Mobile app	Cursor (preview UI)	Cần thấy UI trực quan
Microservices	Claude Code (multi-repo)	Cần làm việc across services

Data Privacy: 3 Mức Độ

3.3.2 — Data Privacy & Compliance

Câu hỏi cốt lõi: *code của bạn đi đâu khi bạn dùng AI tool?*

Mức 1: Cloud Processing

Code gửi lên cloud servers để LLM xử lý. Hầu hết tools mặc định ở mức này. Cần hỏi vendor: code có được lưu trữ? dùng cho training? servers ở đâu?

Mức 2: Zero Data Retention (ZDR)

Code gửi lên, xử lý, trả kết quả, xóa ngay. Không lưu, không log, không training. Anthropic API với ZDR và Cursor với Privacy Mode ở mức này. Đủ cho hầu hết dự án.

Mức 3: 100% Local ("Air-gapped")

Code không bao giờ rời máy. LLM chạy local qua Ollama. Cần thiết cho: dự án quân sự/chính phủ, dữ liệu y tế (HIPAA), tài chính (PCI DSS), hoặc công ty cấm gửi code ra ngoài.

So Sánh Data Privacy Giữa Các Tool

3.3.2 — Data Privacy

Tool	Code dùng training?	Lưu trữ?	Data Region	Certifications
Claude Code (Sub)	Không	30 ngày (trust & safety)	US	SOC 2 Type II
Claude Code (API)	Không	Không (ZDR)	US, EU	SOC 2, HIPAA eligible
Cursor (Pro+)	Không (Privacy Mode)	Không	US	SOC 2
Copilot (Individual)	Có thể (đọc ToS kỹ)	Có thể	US	SOC 2
Copilot (Business+)	Không	Không	US, EU	SOC 2, ISO 27001
Codex CLI (API)	Không	30 ngày	US	SOC 2

⚠️ ⚠️ **Bản Cá Nhân vs Doanh Nghiệp — Copilot:** **Individual:** code có thể được dùng để cải thiện Copilot (training). **Business/Enterprise:** cam kết KHÔNG dùng code cho training. Quyền sở hữu code output thuộc về bạn cả 2 bản — nhưng nếu code AI trùng với code công khai, có thể có vấn đề license.

Khi Nào Cần Local LLM?

3.3.2 — Khi Nào Cần Local LLM

Tình huống	Mức privacy cần	Giải pháp
Đồ án sinh viên thông thường	Mức 1-2 (đủ)	Cursor/Copilot/Claude Code bình thường
Đồ án dùng dữ liệu thực người dùng	Mức 2 (ZDR)	API với ZDR, không paste dữ liệu thực vào prompt
Startup với IP quan trọng	Mức 2-3	ZDR API + local cho core algorithms
Dự án bảo mật cao (quân sự, y tế)	Mức 3 (bắt buộc)	Continue.dev + Ollama + Llama 3
Công ty cấm gửi code ra ngoài	Mức 3 (bắt buộc)	Cline + Ollama hoặc Aider + local model

Stack Bảo Mật Cao — Đề Xuất

VS Code + Continue.dev/Cline + Ollama + Qwen 2.5 Coder 14B

Yêu cầu tối thiểu: 16GB RAM + GPU 8GB VRAM (RTX 3060)

Chiến lược hybrid: 80% local (miễn phí, bảo mật) + 20% cloud API ZDR (tasks khó)

Mô Hình TCO: Chi Phí Toàn Bộ

3.3.3 — Phân Tích Chi Phí

TCO = Subscription + API + Learning + Risk

Subscription Cost

- Cursor Pro: \$20/người/tháng
- Claude Code Max: \$100/người/tháng
- Copilot Individual: \$10/người/tháng

API Cost (Biến Động)

- Claude Sonnet 4.6: \$3/1M in + \$15/1M out
- Claude Opus 4.6: \$15/1M in + \$75/1M out
- GPT-4o: \$2.5/1M in + \$10/1M out
- GPT-4o mini: \$0.15/1M + \$0.6/1M

Learning Cost (Thời Gian)

- Copilot: 1-2 giờ (cài extension xong là dùng)
- Cursor: 2-4 giờ (VS Code familiar)
- Claude Code: 4-8 giờ (terminal workflow mới)
- Cline + Ollama: 4-6 giờ (cấu hình phức tạp)

Risk Cost

- Model-locked: Provider down → không làm việc được
- Model-agnostic: Chuyển provider khác trong 5 phút
- Local LLM: Không phụ thuộc internet

Chi Phí Thực Tế: Nhóm 5 SV · 20 Tuần

3.3.3 — Case Study Chi Phí

Hạng mục	PA-A: Bootstrap (\$0)	PA-B: Standard	PA-C: Hybrid ★ Khuyến Nghị
Subscription/tháng	\$0	5×\$20 = \$100	~\$100
API/tháng	~\$5-10	\$0 (đủ trong sub)	~\$20 (shared key)
Tổng 5 tháng	~\$50	~\$500	~\$600
Tools	Copilot Free + Cline	Cursor Pro only	Cursor + Claude Code
Chất lượng AI	Trung bình	Tốt	Rất tốt
Learning time	6-8h (Cline phức tạp)	2-4h	4-6h
Risk khi down	Thấp (multi-provider)	Trung bình	Thấp (có backup)
Đánh giá	★★★ (tiết kiệm)	★★★★ (đơn giản)	★★★★★ (tối ưu)

 Lưu Ý — Giá API thay đổi thường xuyên và thường giảm theo thời gian. Kiểm tra giá mới nhất trước khi quyết định.



Chiến Lược Multi-Tool

Phần 3.4

Developer giỏi không chỉ dùng 1 tool mà kết hợp nhiều tools cho các mục đích khác nhau.

Giống như thợ mộc không chỉ có 1 cái búa — họ có cả hộp công cụ và biết khi nào dùng cái nào.

Xu hướng rõ ràng trong 2026

Cursor + Claude Code

3.4.1 — Combo Phổ Biến Nhất

Pattern được nhiều developer sử dụng nhất: daily coding với Cursor + complex tasks với Claude Code

Cursor Cho

- Viết code hàng ngày (autocomplete, inline suggestions)
- Small-medium tasks (implement 1 function, fix 1 bug)
- Navigate và hiểu codebase (chat + @codebase)
- Quick refactors trong 1-3 files
- Viết tests cho code vừa viết

Claude Code Cho

- Multi-file refactoring (rename service, thay đổi pattern)
- Greenfield scaffolding (tạo project từ spec)
- Complex debugging (trace bug across 5+ files)
- Architecture changes (thêm caching layer, change DB schema)
- CI/CD setup, Docker configuration
- Code review toàn bộ PR

Copilot (Free) + Cline + Ollama

3.4.2 — Combo Tiết Kiệm

Cho nhóm sinh viên ngân sách \$0

Copilot (GitHub Education = \$0)

- Autocomplete hàng ngày trong IDE
- Chat trong IDE cho câu hỏi nhanh
- Tích hợp sẵn với GitHub Issues, PRs

\$0

Chi phí/tháng

(có GitHub Education)

Cline (Open-source, Free)

- Agentic tasks với local model (Ollama)
- Multi-file edits khi Copilot không đủ
- Dùng API key (pay-per-use) cho tasks khó

Ollama (Free, 100% Local)

- Chạy Qwen 2.5 Coder 14B cho tasks đơn giản
- 100% bảo mật khi làm việc với dữ liệu nhạy cảm

\$5-10

API/tháng

(nếu dùng cloud cho tasks khó)

3 Phương Án: Nhóm 5 SV · 20 Tuần

3.4.3 — Khuyến Nghị Cụ Thể

Phương án	Thành viên	Tool chính	Tool phụ	Chi phí
PA-A Bootstrap	Tất cả 5	Copilot (Free Education)	Cline + Ollama	\$0
	Lead (1)	+ Anthropic API key	Claude Code khi cần	~\$10/tháng
	Tổng: ~\$50 cả 20 tuần			
PA-B Standard	Tất cả 5	Cursor Pro (\$20/tháng)	—	\$100/tháng
	Lead (1)	+ Claude Code Max	—	+\$100/tháng
	Tổng: ~\$1,000 cả 5 tháng			
PA-C Hybrid ★	4 người	Cursor Pro (\$20/tháng)	Copilot Free (backup)	\$80/tháng
	Lead (1)	Claude Code Pro (\$20)	Cursor Pro	\$40/tháng
	Cả nhóm	Anthropic API (shared)	Cho tasks phức tạp	~\$20/tháng

📌 Tổng: ~\$700 cả 5 tháng — TỐI ƯU CHI PHÍ vs HIỆU QUẢ

Tóm Tắt Chương 3

Tóm Tắt — Chương 3

3.1 — Phân Loại Theo Nguyên Lý

3 triết lý tích hợp (IDE-native, Plugin, Terminal-native) + 2 trục phụ (Model-Locked vs Agnostic, RAG vs Long Context). Mọi tool là biến thể của 3 triết lý này.

3.2 — Đánh Giá Chi Tiết

Claude Code mạnh nhất về reasoning. Cursor mượt nhất về flow. Copilot rộng nhất về ecosystem. Config files: dùng AGENTS.md làm nguồn sự thật duy nhất.

3.3 — Bảng Quyết Định

Decision Matrix 6 chiều (quy mô, ngân sách, kỹ năng, loại dự án). Data Privacy 3 mức: Cloud → ZDR → Local. Local LLM với Ollama + Cline cho bảo mật cao.

3.4 — Chiến Lược Multi-Tool

Multi-tool là xu hướng 2026. Combo tốt nhất: Cursor + Claude Code. Combo \$0: Copilot + Cline + Ollama. Khuyến nghị nhóm 5 SV: PA-C Hybrid ~\$700/5 tháng.

Bài Tập Về Nhà

Bài Tập — Chương 3

#	Bài tập	Thời gian	Deliverable
HW-7	Mỗi người cài và thử 1 tool chưa dùng (30 phút). Viết nhận xét 5 dòng.	30p	Tool review (5 dòng)
HW-8	Nhóm chọn tool stack cho đề án. Viết Decision Record.	20p	DECISIONS.md: D-00X
HW-9	Setup AGENTS.md + .cursorrules (hoặc tương đương) cho dự án.	30p	Config files committed
HW-10	(Nếu có máy mạnh) Thử Cline + Ollama local. So sánh với cloud.	40p	Báo cáo local vs cloud

Checklist Hoàn Thành Chương 3

- ☐ Biết 3 triết lý tích hợp và phân loại được bất kỳ tool mới
- ☐ Hiểu Model-Locked vs Agnostic và tác động đến chi phí/bảo mật
- ☐ Đã thử ít nhất 2 tools khác triết lý
- ☐ Chọn được tool stack cho đề án và ghi vào Decision Log
- ☐ Tạo được AGENTS.md + tool-specific config files
- ☐ Hiểu 3 mức Data Privacy và khi nào cần Local LLM
- ☐ Biết setup Ollama + Cline cho local coding (nếu cần)